Report

Part 1: Empirical Coding Task

1. Experimental Setup

I implemented the full attack-detection-repair pipeline on **CIFAR-10** using a **ResNet-18** classifier, following the assignment specification. I chose **class 0** as the target class for the all-to-one backdoor attack. From the CIFAR-10 training set, I first created a **balanced clean validation split** by holding out **500 images per class**, giving a validation set of **5000** images in total. The remaining **45,000** training images were used for model training and poison construction.

For the attack, I used a **BadNets-style fixed trigger**: a **3×3 white square** placed in the **bottom-right corner** of the image with a **1-pixel offset from the border**. I poisoned **5% of the non-target training examples**. Since class 0 was the target class, only non-target examples were poisoned; each poisoned sample was stamped with the trigger and relabeled to class 0. This resulted in **2025 poisoned training samples** out of **40,500 non-target training examples**. This design was intentionally simple, localized, easy to describe, and strong enough to produce a visible backdoor effect without requiring extensive tuning.

For Activation Clustering, I followed the assignment's recommended pipeline. I extracted **penultimate-layer features** from the trained poisoned model, but only for training examples whose **observed label was the target class**. I then **standardized** these features, reduced dimensionality using **FastICA to 10 dimensions**, and applied **k-means clustering with 2 clusters**. To determine which cluster was suspicious, I used the **smaller-cluster heuristic**, since the poisoned points were expected to form a minority group within the observed target-class subset.

For repair, I removed all samples assigned to the suspicious cluster and then retrained a **fresh ResNet-18 from scratch** on the filtered dataset. I then evaluated the repaired model on both the clean CIFAR-10 test set and the triggered non-target test set used for ASR evaluation.

---

2. Task 1A: Backdoor Attack Results

The clean baseline model achieved a **clean test accuracy of 93.27%**. Its **ASR** on triggered non-target test images was only **0.76%**, which shows that the trigger by itself did not naturally force predictions into the target class.

The poisoned model achieved a **clean test accuracy of 92.58%**, which is only slightly lower than the clean baseline. However, its **ASR** increased dramatically to **98.06%**. This means the poisoned model still behaved almost normally on clean test data, but when the trigger was

added to non-target test images, it predicted the attacker-chosen target class almost all the time.

Attack Evaluation Table

Model	Clean Test Accuracy	ASR
Clean baseline	93.27%	0.76%
Poisoned model	92.58%	98.06%

These results indicate that the backdoor attack was highly successful. The attack preserved high clean accuracy while achieving an extremely high ASR, which is exactly the pattern expected from a successful backdoor. This shows why clean test accuracy alone is not sufficient to assess model safety under data poisoning.

---

3. Task 1B: Activation Clustering Results

After training the poisoned model, I extracted penultimate-layer features for all training samples whose **observed label was the target class**. This produced a feature set of **6525 samples**. Out of these, **2025 were truly poisoned**, since poisoned non-target samples had been relabeled to the target class.

I standardized the features, reduced them with **FastICA**, and clustered them into **two groups using k-means**. The resulting cluster sizes were:

- **Cluster 0:** 4527 samples
- **Cluster 1:** 1998 samples

Using the smaller-cluster heuristic, I marked **Cluster 1** as suspicious.

Poison Detection Metrics

Metric	Value
Precision	99.85%
Recall	98.52%
F1 score	99.18%

These results show that Activation Clustering worked extremely well in this setting. The precision is almost perfect, which means nearly all samples flagged as suspicious were actually poisoned. The recall is also very high, which means only a very small number of poisoned samples were missed. Overall, the F1 score confirms that the poisoned and clean target-labeled examples were strongly separable in representation space.

The clustering visualization further supports this result by showing that the target-labeled feature embeddings form two distinguishable groups when projected into 2D and colored by cluster assignment.

---

4. Task 1C: Repair Results

To repair the model, I removed all samples assigned to the suspicious cluster. This removed **1998 samples** from the poisoned training set. I then retrained a new ResNet-18 **from scratch** on the filtered dataset.

The repaired model achieved a **clean test accuracy of 92.62%** and an **ASR of 1.91%**.

Clean vs Poisoned vs Repaired Comparison

Model	Clean Test Accuracy	ASR
Clean baseline	93.27%	0.76%
Poisoned model	92.58%	98.06%
Repaired model	92.62%	1.91%

These results show that the repair step was highly effective. The poisoned model had an ASR of 98.06%, but after filtering suspicious samples and retraining, the ASR dropped to 1.91%. At the same time, the repaired model preserved almost all of the clean accuracy of the poisoned model and remained close to the clean baseline. So, in this experiment, Activation Clustering followed by filtering and retraining significantly reduced the backdoor effect while preserving most of the model's standard performance.

---

5. Required Visualizations

The report includes the following required visualizations:

1. **Attack Summary Plot**

   A bar chart comparing **clean accuracy** and **ASR** for the **clean baseline** and **poisoned model**.

2. **Clustering Visualization**

   A 2D projection of the target-labeled feature embeddings, colored by **cluster assignment**.

3. **Repair Summary Plot**

   A bar chart comparing **clean accuracy** and **ASR** across the **clean baseline**, **poisoned model**, and **repaired model**.

---

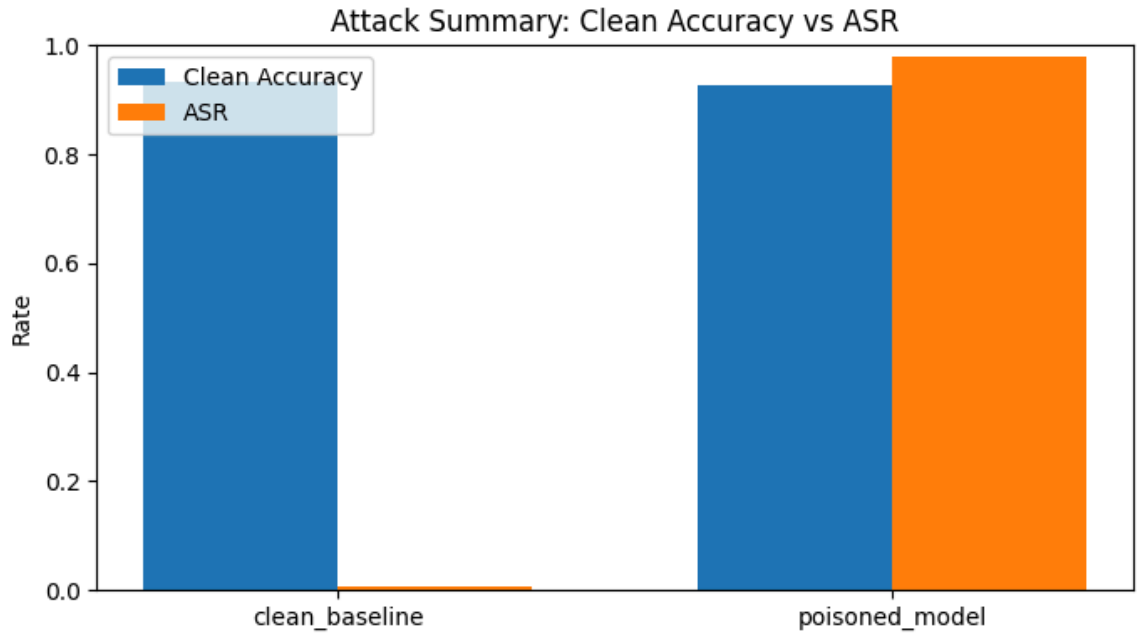
6. Brief Summary of Empirical Findings

This experiment demonstrates the full attack-and-repair pipeline clearly. First, the BadNets-style poisoning attack was highly effective: the poisoned model maintained high clean test accuracy while achieving a very high ASR. Second, Activation Clustering on penultimate-layer features of target-labeled samples was extremely successful at separating poisoned and clean examples, as shown by the very high precision, recall, and F1 score. Finally, removing suspicious samples and retraining from scratch reduced ASR from **98.06%** to **1.91%** while

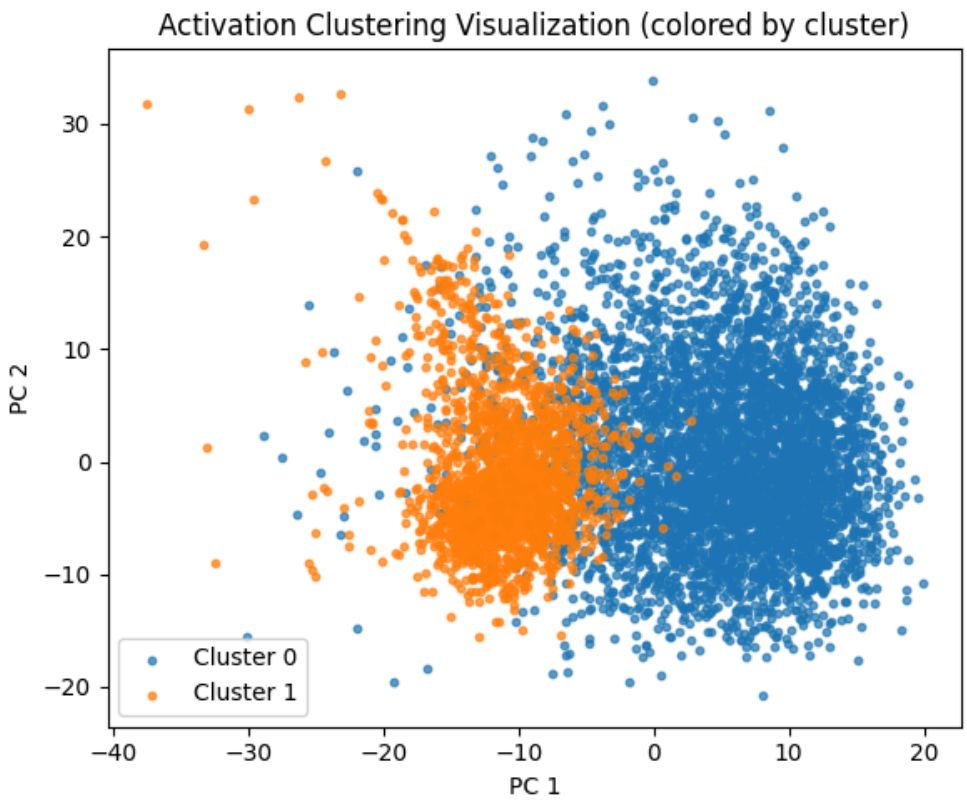
preserving most of the clean accuracy. Overall, the results show that hidden-representation structure can be very useful for detecting and mitigating simple backdoor attacks in a controlled setting.

Visualizations

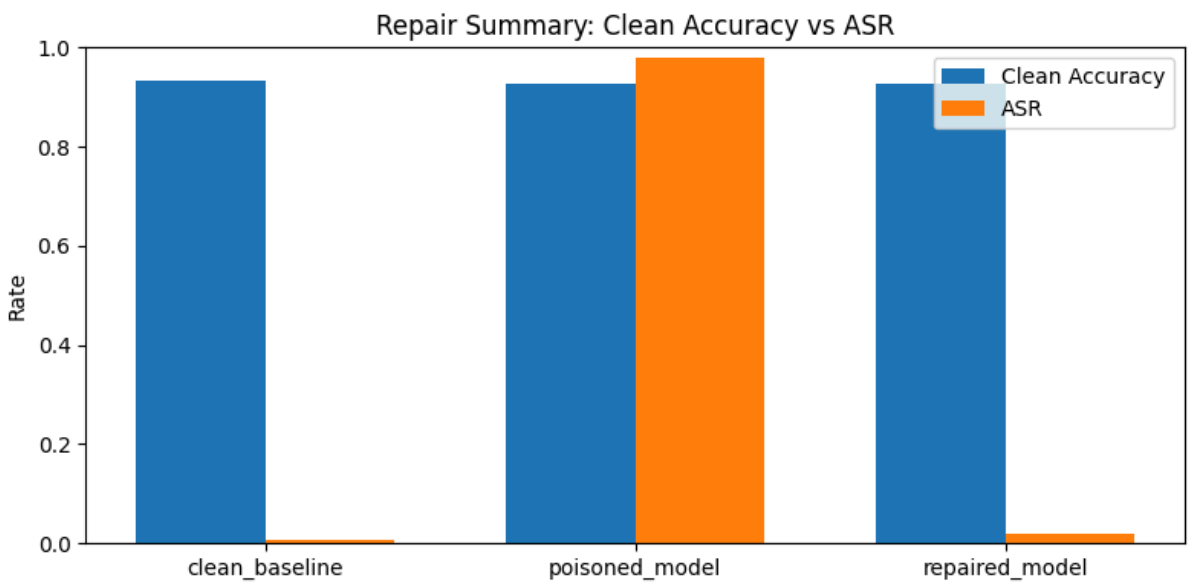
- `attack_summary.png`



- `clustering_visualization.png`



- `repair_summary.png`



Part 2: Theoretical Foundations and Critical Reasoning

1. Why Backdoors Are Dangerous

In my view, **clean test accuracy alone is insufficient** because it only measures average performance on standard, unmodified inputs, while a backdoor is specifically designed to remain dormant unless a particular trigger is present. In other words, a poisoned model can behave normally on almost all clean test images and still contain a highly reliable hidden failure mode. That is exactly what makes backdoors dangerous: they are not meant to hurt ordinary benchmark performance; they are meant to create a **conditional vulnerability** that activates only under attacker-chosen conditions. The assignment itself introduces the problem in those terms: a model can look "perfectly healthy" under ordinary test accuracy and yet still contain a hidden trigger-driven failure mode.

My own results make this very clear. The clean baseline achieved **93.27%** clean test accuracy, while the poisoned model still achieved **92.58%** clean test accuracy, so from a standard benchmark perspective the poisoned model looked almost normal. But when I evaluated on triggered non-target test images, the poisoned model's **ASR jumped to 98.06%**, whereas the clean baseline ASR was only **0.76%**. So the loss in clean accuracy was only about

$$93.27 - 92.58 = 0.69 \text{ percentage points},$$

which is small, but the increase in trigger-induced failure was enormous:

$$98.06 - 0.76 = 97.30 \text{ percentage points}.$$

That gap shows why clean accuracy by itself is a misleading safety metric here. A model can preserve normal utility and still be catastrophically unsafe under a tiny, attacker-controlled perturbation.

I think the deeper issue is that clean accuracy tests the model on the **natural data distribution**, while the backdoor attack creates a highly structured **off-distribution control channel**. Standard evaluation asks, "Does the model classify normal CIFAR-10 images well?" but the safety question is different: "Can the attacker reliably force a chosen output when the trigger is present?" Those are not the same question. A backdoored model looks normal under standard evaluation because the trigger is rare or absent in the clean test set, so the malicious behavior is not exercised. The model is compromised not because it fails on average, but because the attacker can deterministically override its decision rule on demand.

So my conclusion is that evaluating safety under data poisoning requires at least **two** kinds of testing: ordinary clean accuracy to measure standard utility, and some trigger-conditioned metric such as **ASR** to measure whether the model has acquired a hidden attacker-controlled

behavior. My experiments show that a model can score above **92%** clean accuracy and still be almost fully compromised in the backdoor sense.

---

2. Why Hidden Representations Help

I think it is much more sensible to cluster **penultimate-layer activations** than raw pixels because poisoned and clean examples that share the same observed label can still look very diverse in pixel space, while the network's hidden representation compresses them into a more task-relevant space. Intuitively, raw pixels contain a huge amount of irrelevant variation: background, pose, texture, lighting, and class-specific appearance differences. In CIFAR-10, even two clean images from the same class can be very far apart in pixel space for reasons that have nothing to do with poisoning. So if I cluster raw pixels, the clustering can easily be dominated by superficial visual variation rather than the backdoor mechanism itself. The assignment's own defense pipeline reflects this by explicitly asking for clustering on **penultimate-layer features**, not on raw inputs.

The penultimate layer is better because it is closer to the network's internal notion of class structure. By that stage, the model has already transformed the raw image into a representation that suppresses many nuisance details and emphasizes features that matter for prediction. In my setting, I only clustered examples whose **observed label was the target class**. That subset contained two groups: genuinely clean class-0 examples, and poisoned non-target examples that had been relabeled to class 0. In pixel space, those poisoned examples still visually resemble many different source classes, so the geometry is messy. But in hidden space, the backdoor can induce a more coherent representational signature, because the network has learned to associate the trigger with the target label. That is exactly why Activation Clustering is based on hidden activations.

From a representation-learning perspective, the penultimate layer is where the network is trying to create a feature space in which the final classifier can separate classes with a relatively simple decision boundary. If poisoning succeeds, then the network effectively learns a shortcut like: "if trigger is present, map this sample toward the target-class decision region." That means poisoned examples with the observed target label may occupy a different subregion of feature space than genuine target-class examples. In my experiment this is exactly what happened: among the **6525** target-labeled samples I clustered, the method found clusters of size **4527** and **1998**, and the smaller cluster aligned extremely closely with the **2025** truly poisoned samples, producing **99.85% precision** and **98.52% recall**. That kind of near-separation would have been much less likely if I had tried to work directly in raw pixel space.

Another reason hidden representations help is dimensionality and semantics. Raw CIFAR-10 images live in a $32 \times 32 \times 3 = 3072$-dimensional pixel space, where Euclidean distance is often a poor proxy for semantic similarity. My penultimate features were 512-dimensional, and I then standardized them and reduced them to **10 dimensions** using FastICA before clustering, exactly along the lines suggested by the assignment. That sequence makes sense because it moves from a low-level perceptual space to a high-level semantic space and then to a compact

clustering-friendly space. So both intuitively and theoretically, penultimate activations are a much better substrate for poison detection than raw pixels.

---

3. Assumptions Behind Activation Clustering

I think the smaller-cluster heuristic in Activation Clustering works only because several assumptions are simultaneously true. The first assumption is that the poisoned examples form a **minority** within the class-conditioned subset I am clustering. In my pipeline, I poisoned only **5%** of non-target training examples, and when I restricted to the observed target class, I ended up with **6525** samples total, of which **2025** were truly poisoned. So the poisoned points were not the majority; they were a distinct minority mixed into a larger set of genuine target-class examples. That minority structure is crucial because my heuristic literally picks the **smaller cluster** as suspicious. If the attack were much larger, that assumption could fail immediately.

The second assumption is that poisoning produces a **separable representation pattern** in the penultimate layer. In other words, the trigger must induce some relatively consistent internal behavior so that poisoned samples are not just randomly scattered among clean target-class samples. My results suggest that this assumption held very strongly: the suspicious cluster had size **1998**, very close to the **2025** true poisoned samples, and this gave almost perfect precision and recall. But this success depends on poisoned examples forming their own recognizable mode in representation space. If poisoned points overlap heavily with clean target-class points, then k-means has no reason to isolate them.

The third assumption is that the clean target class itself is not already heavily multimodal in a way that confuses clustering. K-means with $k = 2$ assumes that splitting the target-labeled feature cloud into two groups is meaningful, and my heuristic assumes that one of those groups corresponds to poison. But a real class can have multiple natural subclusters because of viewpoint, color, background, or other latent factors. If that natural multimodality is strong enough, clustering may separate two legitimate clean subgroups rather than separating clean from poisoned. So the defense relies not just on poison separability, but also on the clean class being relatively well-behaved in the chosen feature space.

Two clear failure settings come to mind.

First failure case: a semantic or source-specific trigger.

If the trigger is not a fixed visible patch but a high-level semantic feature, such as “all green cars” or “all images from a particular source device,” then poisoned examples may already look semantically compatible with the target class distribution. In that case, penultimate features may not produce a small anomalous cluster at all. The poisoned points could be absorbed into the normal manifold of the class, so the smaller-cluster rule becomes unreliable.

Second failure case: distributed or low-salience triggers.

If the attacker uses a distributed trigger spread across the whole image, rather than a localized corner patch, the backdoor signal may be weakly encoded across many features instead of

creating a sharp, class-conditional cluster. Then poisoned examples may not form a distinct compact group. K-means may either split the data arbitrarily or separate along some unrelated direction.

I can also think of a third one:

Third failure case: high poison rate or adaptive balancing.

If the attacker poisons a large enough fraction of samples, the poison cluster may no longer be the minority. My heuristic explicitly assumes “small suspicious cluster = poison,” so if the attack is large or deliberately balanced, that rule can invert or become meaningless.

So my view is that Activation Clustering is powerful in the regime my experiment used, but it is not a universal detector. It relies on low poison prevalence, class-conditional representational separability, and a clustering geometry where poison stands out as a minority structure. My run worked because those assumptions were approximately true, not because the method is guaranteed to work in every backdoor setting.

---

4. Adaptive Attackers and Stronger Defenses

If the attacker becomes more sophisticated, I think the first part of my pipeline that breaks is the assumption that poisoned points will form a small, isolated cluster in the penultimate-layer feature space. My current defense depends on the chain

$$\text{poison} \to \text{distinct hidden representation} \to \text{separable cluster} \to \text{small suspicious group}.$$

A semantic trigger, source-specific trigger, or distributed trigger attacks this chain directly. Instead of producing a compact “patch-triggered” subpopulation, the attacker tries to make poisoned points blend into the normal class representation. Once that happens, the clustering stage stops being reliable. The assignment itself hints at this by asking how the pipeline would behave under semantic, source-specific, or distributed triggers; those are exactly the cases where a simple visible-patch defense is least robust.

More specifically, I think **three components** are especially fragile.

First, the target-class-only clustering step.

Right now I cluster only examples whose observed label equals the target class. That works when relabeled poisoned examples form a strange minority inside that class. But if an adaptive attacker makes poisoned examples look more like natural members of the target class in feature space, this restriction actually hides the problem by forcing me to search for poison only within a distribution where the poison is designed to blend in.

Second, the $k = 2$ k-means assumption.

K-means assumes roughly convex cluster structure and forces the data into two groups whether or not that partition is meaningful. Under more adaptive attacks, the poisoned samples may not

form one compact group at all; they may be diffuse, manifold-like, or entangled with clean points. Then the 2-cluster decomposition is too rigid.

Third, the smaller-cluster heuristic.

This is probably the easiest component for an adaptive attacker to break. If the attacker increases poison prevalence, distributes the trigger, or intentionally causes poisoned and clean points to mix, then “smallest cluster = poison” becomes a weak guess rather than a principled detector.

One concrete way I would strengthen the defense is to replace the pure size-based clustering heuristic with an **exclusionary retraining / relabeling test** on clusters. The idea is: after clustering the target-labeled features, I temporarily remove one cluster, retrain or fine-tune a lightweight classifier, and then test whether the removed samples still strongly map to the target class. If one cluster behaves like mislabeled or shortcut-driven data, that is stronger evidence of poisoning than just saying “it is smaller.” This improves the defense because it adds a **behavioral validation step**, not just a geometric one. Geometry alone can be fooled; geometry plus label-consistency testing is harder to fool.

The reason I like this strengthening is that it directly tests the hypothesis I actually care about: not merely whether a group is unusual, but whether it is unusual in a way that is consistent with poisoning. In my run, the smaller-cluster heuristic already worked very well, giving **99.18% F1**, but the repaired model still had **1.91% ASR**, not the clean model’s **0.76%**, which suggests that some poisoned influence remained. That residual gap is exactly why I think a stronger post-clustering validation step would be useful. It could help catch the few poisoned points that size-based clustering missed.

So my final view is that my current pipeline is strong against a simple BadNets-style patch trigger, but its weakest link is the assumption that poison leaves behind a clean minority cluster. A stronger defense should not rely only on cluster size. It should combine representation-space anomaly detection with some direct test of whether the suspicious samples are causally responsible for the backdoor behavior.